

App Development Project Report

1. Student Details

Name: Grishma Nikunj Mehta

Roll Number: 22f1000571

Email: 22f1000571@ds.study.iitm.acin

About Me: I am currently pursuing my Master's degree in Applied Statistics and Analytics at NMIMS, Mumbai, and have completed Bachelor's in Statistics at Mithibai College, Mumbai. My academic focus lies in statistical modelling, data analytics, and applied machine learning, with hands-on experience in tools such as Python, R, SQL, Tableau, and Power BI. I enjoy applying analytical techniques to real world problems and building data driven applications that integrate statistics with technology.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

The objective of this project is to design and develop a web-based placement portal that connects students and companies for recruitment processes. The platform allows companies to post placement drives, students to apply for jobs, and administrators to manage the entire system.

Approach:

The app was built using the Flask web framework with a modular architecture. The system supports role based access control for three types of users: Admin, Company, and Student. Students can create profiles, upload resumes, search for jobs, and apply for placement drives. Companies can create job postings and review applicants. The administrator oversees the entire system, approving companies and placement drives and managing users.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist in designing UI styling for HTML templates, debugging flask errors and routing issues, structuring the database schema, improving frontend layout and styling. The extent of AI usage is approximately 40-45%, mainly for debugging assistance and UI styling suggestions. All application logic, database design, feature implementation, testing, and integration were implemented manually.

4. Technologies and Frameworks Used

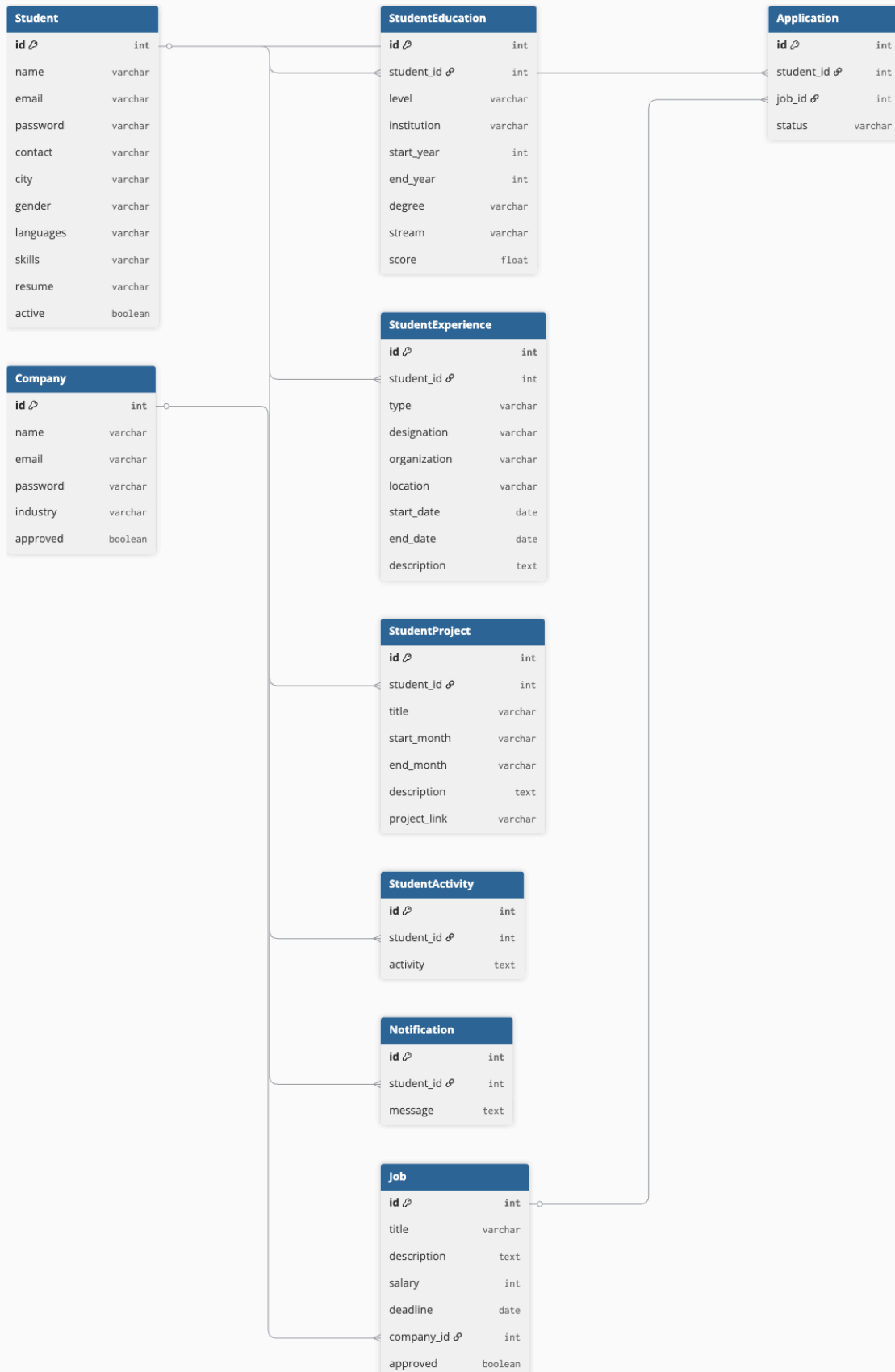
Technology / Library	Purpose
Flask	Backend web framework
SQLAlchemy	Object Relational Mapper for database modeling
Jinja2	Dynamic HTML template rendering
HTML	Structure of frontend pages
CSS	Styling and layout of the application
Flask-Login	User authentication and session management
JavaScript	Client side interactivity
SQLite	Database for storing application data

These technologies were selected because Flask provides a lightweight framework suitable for building modular web applications with integrated database support.

5. Database Schema / ER Diagram

Relationships:

- One-to-Many → *Student* → *Application*
- One-to-Many → *Company* → *Job*
- One-to-Many → *Job* → *Application*
- One-to-Many → *Student* → *Education/Experience/Projects/Activities*



Tables:

1. **Student** — stores student profile details. (id-primary key, name, email, password, contact, city, gender, languages, skills, resume, active)
 2. **Company** — stores company account information. (id, name, email, password, industry, approved)
 3. **Job** — represents placement drives created by companies. (id, title, description, salary, deadline, company_id, approved)
 4. **Application** — stores job applications submitted by students. (id, student_id, job_id, status - Applied, Shortlisted, Selected, Rejected)
 5. **StudentEducation** — stores student educational qualifications.
 6. **StudentExperience** — stores job and internship experience.
 7. **StudentProject** — stores academic and personal projects.
 8. **StudentActivity**— stores extracurricular activities
-

6. API Resource Endpoints

The application primarily uses server side routes implemented in Flask

Endpoint	Method	Description
<code>/login</code>	POST	Authenticate user
<code>/register_student</code>	POST	Register student account
<code>/register_company</code>	POST	Register company
<code>/create_job</code>	POST	Company creates placement drive
<code>/apply_job/<id></code>	GET	Student applies for job
<code>/view_applications/<job_id></code>	GET	Company views applicants
<code>/shortlist/<id></code>	GET	Shortlist applicant
<code>/select/<id></code>	GET	Select applicant
<code>/reject/<id></code>	GET	Reject applicant

7. Architecture and Features

Core Features Implemented:

- Admin: Approves companies and placement drives, monitors system statistics, searches users, and manages student accounts.
- Company: Creates placement drives, view applicants, and updates application status (shortlisted, selected, rejected).
- Student: Builds a profile, uploads resume, searches and applies for jobs, and tracks application status.

Project Structure:

```
▼ PLACEMENT-PORTAL
  > __pycache__
  > instance
  > static
  ▼ templates
    <> add_activity.html
    <> add_education.html
    <> add_experience.html
    <> add_project.html
    <> admin_dashboard.html
    <> company_dashboard.html
    <> create_job.html
    <> edit_profile.html
    <> login.html
    <> register_company.html
    <> register_student.html
    <> student_dashboard.html
    <> student_profile.html
    <> update_resume.html
    <> view_applications.html
  ◆ .gitignore
  📄 app.py
  📄 models.py
  ⓘ README.md
  ≡ requirements.txt
```

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1L3vptT6rytfiaeUZIYM68eOvaMVz2UbG/view?usp=sharing>
